

Saturated-Lattice Bounds and Source-Anchored Primitive Projections

Polynomial Raw Epochs in Exact Unknot Recognition

A research continuation for the ProveIt project
Prepared with ChatGPT; mathematical and integration review invited

8 October 2026

Abstract

The maintained ProveIt recognizer supports exact primitive-pair quotients and unit-coordinate forest contractions on compressed group presentations. Its current analysis bounds consecutive raw phases by recurrences exponential in the phase count. We replace that accounting by a global invariant of a frozen source presentation. Selected original exponent-sum rows span the complete annihilator of the cumulative monomial quotient; each quotient column is a primitive integer kernel vector with disjoint support. Cofactors and Hadamard's inequality then bound every cumulative exponent by $(t-1)^{(t-1)/2}L^{t-1}$ on a block of $t \geq 2$ original generators; singleton columns have unit entries. All exponent bit lengths are consequently $O(r(B + \log(r+1)))$, independently of raw phase depth. A reachable-versus-allocated circuit analysis shows that even the existing raw updater has polynomial total cost throughout such an epoch.

We also implement a source-anchored kernel which evaluates signed counts and primitive-width profiles on one immutable source circuit, rather than repeatedly substituting into historical circuits. It includes independent source-bound replay, bounded certificate parsing, an adapter seam, 38 passing tests, and reproducible exact audits. Completed audits include 26,037 raw-word comparisons and a 64,513-bit cumulative exponent. Paired measurements isolate maintenance of supplied schedules; eager materialization is a substantial regression. The results remove a raw-depth obligation, not the need to expose useful relations or control representation resets. They do not establish general quasi-polynomial unknot recognition, and no complete native recognizer integration or difficult-knot speedup is claimed.

Deliverable status. Proved local complexity improvement; standalone exact research implementation; checked algebraic fixtures; proposed native integration. Not a replacement for source-topology verification.

Contents

1	Research objective and contribution boundary	3
2	The frozen-source epoch model	4
3	A global lattice invariant	6
4	Cofactor bounds and their consequences	7
5	Reachable circuits versus historical allocation	9
6	Source-anchored evaluation without historical substitution	10
7	Independent replay and the nonabelian safety boundary	13
8	Reproducible validation	14
9	Paired maintenance measurements and a negative result	16
10	Integration plan for the maintained repository	17
11	A more precise route to quasi-polynomial recognition	18
12	Further research questions and concrete targets	19
13	Conclusion	20
A	Proof of the coherent-width certificate	21
B	A worked three-generator epoch	22
C	Package map and reproduction commands	22

Reading guide. Sections 3–5 contain the main proof and the sharpened analysis of the maintained updater. Section 6 gives the new representation and its bit-complexity contract. Sections 7–9 specify the independent checker and distinguish completed exact audits from narrowly scoped timings. Sections 10–12 describe native integration, the conditional global complexity statement, and the remaining research tasks. The appendices provide the primitive-width proof, a worked example, and reproduction commands.

1 Research objective and contribution boundary

The objective is to advance the actual complexity analysis and implementation of the ProveIt unknot-recognition program, rather than replace it with a fast but incomplete invariant test. The particular bottleneck addressed here is the repeated manipulation of compressed group relators after verified primitive-factor quotients. The repository already has these algebraic moves, source-bound replay, and an optional producer; the new contribution is a stronger global size argument and a representation that exploits it.

The preceding maintained analyses, in `synthesis/primitive_projection.tex` and `synthesis/primitive_forest.tex`, are careful to separate raw phases from normalization and other search operations [3, 4]. Their bounds are valid upper bounds, but they need not be sharp. In particular, an inequality of the form $B_{j+1} \leq 2B_j + O(1)$ does not show that a valid execution can double its exponent bit length at every step. The successive exponents are constrained by a single initial matrix, and that constraint is absent from the local recurrence.

1.1 What is strengthened

The main theorem proves a polynomial bit-size bound for an entire *frozen-source raw epoch*. The permitted operations are consecutive primitive-pair contractions, including disjoint batches and the leaf-first linearization of unit-coordinate forests. Donors are always the current images of fixed original relator slots; no arbitrary relator replacement is allowed inside the epoch. Rank falls at every contraction, but no logarithmic or polylogarithmic bound on the number of phases is needed.

There are two distinct consequences. First, a sharper analysis applies to the existing raw circuit updater: current reachable nodes need not double when historical allocated nodes do. Second, a new source-anchored representation avoids much of the remaining historical rebuilding. The first is a theorem about a maintained mechanism. The second is implemented here as an isolated research kernel, not silently installed into the full recognizer.

1.2 Relationship to the general target

A quasi-polynomial unknot-recognition algorithm was announced in 2021, following Lackenby’s UC Davis lecture [8]. His July 2026 manuscript develops hierarchies and an unknot algorithm; Section 9 does not give running-time bounds for all of its operations, and the discussion still requires control of hierarchy parameters [9]. The maintained ProveIt program likewise does not claim an implemented general quasi-polynomial bound [1]. This note addresses a specific algebraic component of that implementation program, not a replacement proof of the announced geometric algorithm.

Compressed power-word algorithms provide a broader context for manipulating very large powers without expansion [11]. We do not import an unproved cost bound for normalization from that literature. The raw kernel uses integer arithmetic, fixed-source circuit scans, and exact prefix extrema; it needs no compressed word-equality oracle. Primitive words in rank two have classical structure [10]; the repository’s coherent-width criterion is reused and proved in the form needed here.

1.3 Claims ledger

Claim	Status and precise scope
Global cumulative exponent bound	Proved for frozen-source monomial primitive contractions.
Polynomial total raw-epoch cost	Proved, including a sharper analysis of the existing raw updater.
Source-anchored pair implementation	Implemented and independently replayed in the supplied package.
Source-anchored forest producer	Covered by the mathematical reduction; not implemented as a new forest API here.
Measured maintenance improvement	Completed on supplied algebraic schedules; discovery and replay excluded from timers.
Whole-recognizer speedup	Not established by these experiments.
General quasi-polynomial recognition	Not established; exposure, resets, and complete search remain separate obligations.

2 The frozen-source epoch model

Let $F(X)$ be the free group on $X = \{x_1, \dots, x_r\}$, where $r \geq 1$. Actual implementation labels may be any distinct positive integers. An epoch starts with a fixed presentation

$$G = \langle x_1, \dots, x_r \mid R_1, \dots, R_m \rangle.$$

Each R_j is supplied by a binary straight-line program (SLP): an acyclic circuit whose leaves are signed generators and whose internal gates concatenate their two inputs. Gate zero represents the empty word. A root may be repeated, and slots may initially be empty. An SLP represents a *literal word*, not its freely reduced equivalence class.

Write M for the number of frozen source gates, including zero, and

$$L = \max\{1, |R_1|, \dots, |R_m|\}.$$

For processing bounds let $B \geq 1$ dominate the bit length of the length of *every scanned source gate*. If the source is trimmed to the union of its root-reachable gates, one may take $B = \lceil \log_2(L + 1) \rceil$. The supplied snapshot operation performs this trimming. Arbitrary serialized sources may contain unused gates; then their length metadata must also be charged. Since a binary circuit with M gates has represented-length bit size $O(M)$, this qualification does not destroy polynomial input-bit complexity.

Definition 2.1 (Raw epoch). A raw epoch retains the original relator slots and modifies them only by a cumulative homomorphism induced by the permitted primitive contractions. A slot used as a donor is marked deleted and represented by the empty root. Every other slot remains the current literal image of its original root. There is no intervening arbitrary basis change, nonmonomial substitution, relator recombination, normalization, or source reset.

This definition is intentionally narrow. At a normalization or exposure boundary, the producer may freeze a new source and begin a new epoch, but its construction and size must be charged separately. Treating a changed source as though it were still the old one would invalidate the matrix argument below.

2.1 A permitted primitive-pair contraction

Suppose a current relator is w^e , where $e \geq 1$, w is primitive in $F(a, b)$, and its signed exponent-sum vector is (u, v) with $uv \neq 0$. Primitivity implies $\gcd(|u|, |v|) = 1$. We use the convention $a < b$ and reuse the name a for a *new* quotient generator:

$$a \mapsto a^{|v|}, \quad b \mapsto a^{-u \operatorname{sgn}(v)}. \quad (1)$$

The right-hand occurrence of a is not recursively substituted into itself. All other live generators are unchanged.

Lemma 2.2 (The local quotient). *The quotient $F(a, b)/\langle\langle w \rangle\rangle$ is infinite cyclic, and (1) realizes it. Substitution into every retained relator therefore computes the quotient of the complete presentation by $w = 1$. If $e = 1$, this preserves the presented group. If $e > 1$, it preserves the presented group whenever that group is torsion-free.*

Proof. Since w belongs to a free basis of $F(a, b)$, killing it leaves a free group of rank one. The character in (1) annihilates w and is onto, because its two image exponents are coprime. It induces a surjection between two infinite cyclic groups and hence an isomorphism. For $e > 1$, the already imposed relation $w^e = 1$ implies $w = 1$ in a torsion-free group. In both cases adding $w = 1$ changes no group element's equality class; rewriting the quotient free group and every remaining relator is a presentation transformation. $\square$

For a validated classical knot-group source, torsion-freeness is the same source-derived premise used by the maintained primitive modules [3, 4]. The research API does not infer it from an arbitrary presentation. Its verifier reports whether proper powers occurred; a caller-supplied Boolean is not a topological proof.

Disjoint pairs can be validated against the same pre-batch state and applied simultaneously, because they affect disjoint free factors. Selected source slots are then tombstoned. Although a selected donor's raw image may contain opposite letters, its one-generator exponent is zero, so deleting precisely that slot is justified. Unselected slots are never dropped by inference.

2.2 Forests and their linearization

A unit-coordinate primitive donor has an equation of the form $c = p^k$ with nonzero integer k . Select at most one parent for each child, and require the dependency graph to be a forest. The maintained forest operation can substitute all dependencies in one batch [4].

Lemma 2.3 (Forest linearization). *A valid unit-coordinate forest batch has a sequential realization by primitive-pair eliminations that use the same original donor slots, retain the intended anchors, and yield the same monomial image of each initial generator. Consequently all results below apply to arbitrary mixtures of pair batches and forest batches within a raw epoch.*

Proof. Eliminate a leaf child c first, keeping its parent p . No remaining selected donor has c as its parent, so none of those donor words changes at that step. Continue leaf-first. Each later selected donor is therefore still its pre-batch word when used. The image of a vertex is the anchor raised to the product of edge powers on its path, exactly as in simultaneous forest substitution. Orienting a cyclic quotient to keep the parent instead of the smaller numerical label changes only generator naming and possibly its sign; neither affects the bounds. $\square$

The acyclic and unit-coordinate hypotheses matter. Overlapping arbitrary primitive pairs need not present a free factor after simultaneous killing, and cycles are not covered by this argument.

3 A global lattice invariant

Index rows by original generators and columns by current live generators. At every raw state the cumulative map is monomial:

$$x_i \mapsto z_{\pi(i)}^{k_i}, \quad k_i \in \mathbb{Z} \setminus \{0\}. \quad (2)$$

Let $K \in \mathbb{Z}^{r \times s}$ record this map: row i has its sole nonzero entry k_i in column $\pi(i)$. For a live generator z , let $I_z = \{i : \pi(i) = z\}$ be its block. All blocks are nonempty and partition X .

Let $A \in \mathbb{Z}^{m \times r}$ be the source exponent-sum matrix:

$$A_{ji} = \text{signed exponent sum of } x_i \text{ in } R_j.$$

In particular $|A_{ji}| \leq L$. Let J be the set of original slots selected as donors so far, and A_J its row submatrix. This matrix is an analysis object; the producer need not construct it.

Lemma 3.1 (Primitive disjoint columns). *For each live block I_z ,*

$$\gcd\{|k_i| : i \in I_z\} = 1.$$

In particular, the lattice $\text{im}_{\mathbb{Z}} K$ is a direct summand of $\mathbb{Z}^r$.

Proof. Initially K is the identity. Suppose a contraction merges two primitive columns supported on disjoint blocks. Formula (1) multiplies one block by $|v|$ and the other by $-u \text{sgn}(v)$. The gcd in their union is $\gcd(|u|, |v|) = 1$; other columns are unchanged. Induction proves the first claim. A primitive vector on a coordinate block extends to an integral basis of that block. Taking those extensions independently for all disjoint blocks proves the direct-summand statement. $\square$

Lemma 3.2 (Independence of selected source rows). *After q individual contractions, $s = r - q$ and*

$$A_J K = 0, \quad |J| = q, \quad \text{rank}_{\mathbb{Q}} A_J = q.$$

Proof. Each selected source word is killed by its contraction, so its exponent row annihilates the resulting K and every later cumulative map. This gives $A_J K = 0$.

For independence, consider the next donor row a and the previous map K_{old} . The vector aK_{old} is the current donor's exponent vector: its two nonzero coordinates are eu and ev . It is therefore nonzero. Every previous selected row annihilates K_{old} , so a cannot be in their rational span. Induction gives rank q . Disjoint batches can be linearized, and forests are covered by Lemma 2.3. $\square$

Theorem 3.3 (Saturated kernel identity). *At every state in a raw epoch,*

$$\ker_{\mathbb{Q}} A_J = \text{im}_{\mathbb{Q}} K, \quad \ker_{\mathbb{Z}} A_J = \text{im}_{\mathbb{Z}} K. \quad (3)$$

Moreover the restricted matrix $A_J[:, I_z]$ has a one-dimensional rational kernel, spanned by the primitive column $(k_i)_{i \in I_z}$.

Proof. The columns of K are independent because they are nonzero and have disjoint support. Lemma 3.2 gives $\dim \ker_{\mathbb{Q}} A_J = r - (r - s) = s$, proving the rational identity.

For the integral identity, take $y \in \ker_{\mathbb{Z}} A_J$ and write $y = Kc$ with $c \in \mathbb{Q}^s$. On a block I_z , $y_i = c_z k_i$. A Bezout combination of the k_i equals one by Lemma 3.1; the same combination of the integer y_i equals c_z . Thus $c_z \in \mathbb{Z}$, proving integrality.

Finally, a vector in the kernel of $A_J[:, I_z]$ can be extended by zero outside that block. The rational identity expresses this extension using the columns of K . Disjoint support forces every other column coefficient to vanish, so the restricted kernel is exactly the indicated line. $\square$

This is the key source constraint missing from a stepwise length recurrence. It is essential that the selected rows belong to one fixed source. The argument says substantially more than preservation of abelianization, but it does *not* replace the nonabelian primitive-word checks that made the contractions legitimate in the first place.

4 Cofactor bounds and their consequences

Define

$$\Delta_t(L) = \begin{cases} 1, & t = 1, \\ (t-1)^{(t-1)/2} L^{t-1}, & t \geq 2. \end{cases}$$

Theorem 4.1 (Phase-independent exponent bound). *For every raw state and every block I_z of size t ,*

$$|k_i| \leq \Delta_t(L) \quad (i \in I_z). \quad (4)$$

Consequently every cumulative exponent has

$$O(r(B + \log(r+1))) \quad (5)$$

bits throughout the entire raw epoch. There is no dependence exponential in the number of raw phases.

Proof. For $t = 1$, primitivity gives $|k_i| = 1$. For $t \geq 2$, choose $t-1$ independent rows of $A_J[:, I_z]$ and call the resulting matrix C . Theorem 3.3 shows this is possible and that $\ker C$ is one-dimensional. Let

$$c_i = (-1)^{i+1} \det C_{\hat{i}}$$

be its vector of alternating maximal minors. Laplace expansion gives $Cc = 0$, and row rank $t-1$ ensures $c \neq 0$. Dividing c by the gcd of its entries gives the unique primitive integral kernel vector up to sign. Therefore $|k_i| \leq |c_i|$.

Each row of a $(t-1) \times (t-1)$ minor has Euclidean norm at most $\sqrt{t-1} L$. Hadamard's inequality bounds its determinant by the product of these norms, giving (4). Taking binary logarithms and using $t \leq r$ and $\log_2 L \leq B$ proves (5). $\square$

For later bookkeeping fix the convenient common bound

$$\Lambda = 8(r+1)(B + \lceil \log_2(r+1) \rceil + 2). \quad (6)$$

Constants here are deliberately loose. Cumulative exponents, lengths of substituted source gates, primitive donor coordinates, and intermediate prefix-height quantities have $O(\Lambda)$ bits. For example,

a mapped root has length at most $L \max_i |k_i|$. A donor's primitive coordinates are bounded by its occurrence counts, and its height extrema multiply counts by such coordinates; this only multiplies the bit bound by a constant.

Corollary 4.2 (Polynomial raw depth is harmless). *Any valid raw epoch has at most $r - 1$ individual contractions, and all integer data needed for its source-based primitive checks and monomial updates have polynomial size in the initial encoded parameters. This remains true for $r - 1$ sequential one-pair phases or arbitrarily deep valid forests.*

4.1 Why primitivity, rather than arbitrary scaling, matters

The cofactor argument bounds the *primitive* kernel vector, not arbitrary multiples of it. If a procedure were allowed to replace a live generator by an arbitrary power without maintaining surjectivity, the same rational kernel line could be represented by arbitrarily large integer vectors. The gcd condition is therefore not cosmetic. It is the exact property that prevents spurious scale growth.

Similarly, independent source rows are necessary. Merely accumulating many relations that vanish after projection gives no determinant bound on the map unless they have the full required rank. The nonzero current donor exponent vector supplies that rank increase at each contraction.

4.2 A family requiring linear growth of exponent bit length

Proposition 4.3 (Sharpness of the rB dependence). *For $q = 2^b$ and $r \geq 2$, the source presentation*

$$\langle x_1, \dots, x_r \mid x_i^q x_{i+1}^{-1}, 1 \leq i < r \rangle \quad (7)$$

is infinite cyclic. A valid sequential raw epoch produces $x_i \mapsto x_1^{q^{i-1}}$, and the largest exponent has exactly $(r - 1)b + 1$ bits.

Proof. Each source relation eliminates x_{i+1} as x_i^q by an ordinary Tietze move. Applying these eliminations in increasing order gives the stated powers and leaves the free generator x_1 . The bit count follows from $q^{r-1} = 2^{b(r-1)}$. $\square$

Thus a logarithmic bound in r would be false in general. Nevertheless the bit growth is linear in rB , not exponential in the number of contractions. These are abstract presentations of $\mathbb{Z}$, not claimed difficult knot diagrams. The distinction is especially important for the capacity experiments below.

4.3 A stronger bound from a known nonzero character

Theorem 4.4 (Character-controlled columns). *Let $h \in \mathbb{Z}^r$ satisfy $Ah = 0$. On any block I_z for which h is not identically zero, let $g_z = \gcd\{|h_i| : i \in I_z\}$. Then $h_i \neq 0$ for every $i \in I_z$, and*

$$k_i = \varepsilon_z \frac{h_i}{g_z}, \quad \varepsilon_z \in \{1, -1\}. \quad (8)$$

In particular $|k_i| \leq \max_{i \in I_z} |h_i|$ on such a block.

Proof. Since $A_J h = 0$, Theorem 3.3 gives $h = Kc$ with $c \in \mathbb{Z}^s$. On I_z , $h_i = c_z k_i$. If the restriction is nonzero, $c_z \neq 0$ and all $k_i \neq 0$, so all $h_i \neq 0$. Their gcd is $|c_z|$, because the k_i are primitive. Solving for k_i proves (8). $\square$

For consistently oriented Wirtinger meridians the usual character sends every generator to one. A raw epoch frozen at that presentation therefore has $|k_i| = 1$ throughout. Large powers arise only after a change of source coordinates, or on other supplied presentations. This gives both a useful optimization target and a warning: algebraic huge-power fixtures are not automatically representative of direct meridional inputs. Blocks on which a known character is identically zero still require the general cofactor bound.

5 Reachable circuits versus historical allocation

The preceding result also sharpens the analysis of the existing updater; a new representation is not necessary merely to obtain a polynomial raw-epoch bound. This observation prevents an incorrect exponential-speedup narrative.

Let S_j denote the number of nodes reachable from the current relator roots before batch j , and let N_j be all arena nodes allocated so far. They need not be comparable by a fixed constant: old versions can remain allocated without being current. Let P_j conservatively count all gates needed for signed local power images in that batch, including reused gates participating in those image circuits.

Lemma 5.1 (Reachable-node recurrence). *For the raw mapper that substitutes terminal images and maps each current reachable concatenation once,*

$$S_{j+1} \leq S_j + O(P_j), \tag{9}$$

$$N_{j+1} - N_j \leq S_j + O(P_j). \tag{10}$$

Proof. Each old reachable concatenation has at most one mapped concatenation in the new current graph. Its children are either other mapped nodes or nodes in the chosen terminal-image circuits. Unchanged terminals can remain as before; changed terminals are accounted for among image gates. The new current graph therefore contains at most one mapped representative for each old reachable node, together with the image circuits. Old historical versions do not add a second copy to *current reachability*. Structural interning and deletion of donor roots can only lower the count. The same construction allocates at most that many new nodes, giving the second inequality. $\square$

The pinned `apply_projection` implementation uses exactly this pattern: it constructs binary powers and performs one snapshot traversal of `arena._reachable(roots)` [5]. The inspected word-arena primitives use raw concatenation and binary powering; they do not call normalization in these operations [6]. Negative images are constructed from oppositely signed letters. The forest variant constructs shared parent powers with the same total local-power accounting [4].

There are at most $r - 1$ contractions. Theorem 4.1 bounds every local power exponent by $O(\Lambda)$ bits, so binary powering gives

$$\sum_{j < d} P_j = O(r\Lambda).$$

Combining this with Lemma 5.1 yields the following result.

Theorem 5.2 (Polynomial cost for the existing raw updater). *Throughout an epoch of $d \leq r - 1$ successful raw batches,*

$$S_j = O(S_0 + r\Lambda), \quad (11)$$

$$N_d \leq N_0 + O(dS_0 + (d+1)r\Lambda) \quad (12)$$

$$= N_0 + O(rS_0 + r^2\Lambda). \quad (13)$$

For the inspected raw primitives and polynomial arithmetic donor checks, planning, updating, and replaying the entire epoch have polynomial total bit cost in the initial arena size, initial numeric metadata, m , r , and label bit sizes. Normalization and legacy search are excluded.

Proof. Sum (9) to obtain (11), and then sum (10). Every scan and arithmetic primitive-width check is polynomial in the current reachable or allocated data. There are at most r full candidate scans, counting the final unsuccessful one, and at most $r - 1$ contractions. Exact additions, multiplications, gcd computations, sorting of reachable identifiers, and cached metadata updates all have polynomial cost on the bounded integers. If the epoch starts in an arena containing old unused nodes, their existing size and maximum metadata bit length are part of the initial input cost; the theorem does not erase that history. $\square$

5.1 The correction to the earlier accounting

The prior inequality $N_{j+1} \leq 2N_j + O(rB_j)$ remains true. Its use as the only recurrence is unnecessarily pessimistic because it treats every historically allocated node as though it were copied into every current graph. The preceding two-counter analysis repairs this. Separately, the cofactor bound replaces a pessimistic recurrence for B_j by a single uniform bound. Both improvements are required to obtain the clean epoch statement.

Accordingly, the old $2^{O(d)}$ and $2^{O(d \log r)}$ raw bounds should be *sharpened*, not described as refuted execution lower bounds. A general short-depth producer was never established; this note shows that short raw depth is not the remaining issue under the epoch model. It does not show that a useful raw epoch is easy to expose or that a search uses few source resets.

6 Source-anchored evaluation without historical substitution

The new representation stores the source circuit once and maintains only

$$(\pi(i), k_i)_{i=1}^r, \quad \text{live labels,} \quad \text{deleted source slots,} \quad \text{move records.}$$

A source terminal x_i^ϵ is interpreted as a homogeneous run of $|k_i|$ letters, with current label $\pi(i)$ and sign $\epsilon \operatorname{sgn}(k_i)$. No power circuit is built for that run while the epoch remains raw.

6.1 Signed counts and eligibility

A bottom-up scan assigns a gate either a map from at most two current labels to positive and negative occurrence counts, or a marker meaning that more than two labels occur. Concatenation adds counts and unions supports. Since these are literal words, opposite occurrences are not cancelled. A relator is a candidate precisely when its current support has two labels, each occurring with a single sign.

The scan must be recomputed for a new image table. An old source gate that once used three distinct live generators may use only two after contractions. Caching its “more than two” marker independently of the image state would be unsound. The native updater avoids that issue by assigning new node identities to changed words; a fixed-source implementation must instead use state-sensitive summaries. The reference kernel deliberately recomputes them.

Let a coherent candidate have signed counts $U, V \neq 0$. Put

$$e = \gcd(|U|, |V|), \quad u = U/e, \quad v = V/e, \quad p = |u|, \quad q = |v|.$$

Rename or invert its two generators so its word is positive. The primitive width test uses an increment q on its first letter and $-p$ on its second.

6.2 Prefix extrema on a fixed circuit

For a word segment W , store the triple (D, ℓ, h) of total height change, minimum prefix height, and maximum prefix height, including the empty prefix. For two segments the exact composition law is

$$(D, \ell, h) \star (E, \ell', h') = (D + E, \min(\ell, D + \ell'), \max(h, D + h')). \quad (14)$$

A homogeneous source-leaf run with total increment d has profile $(d, \min(0, d), \max(0, d))$. Thus internal points in a huge power require no iteration: a constant-slope segment attains its extrema at its endpoints.

Proposition 6.1 (Coherent-width certificate). *Let W be a coherent two-letter word with counts ep, eq , where $p, q > 0$ and $\gcd(p, q) = 1$. If the profile with increments $q, -p$ has width $p + q - 1$, then W is the e th power of a cyclically shifted primitive word. The condition can be checked exactly by (14) on the source SLP.*

Appendix A gives a self-contained proof of the sufficient criterion. This is the criterion already used by the maintained primitive module [7]; the present work changes its evaluation representation and global complexity accounting, not its logical role. When one total occurrence count is one, the word is automatically primitive: it is cyclically a power of one generator followed by a single occurrence of the other. The producer uses this shortcut; the independent verifier does not.

6.3 Updates and a finite producer

For a disjoint batch, form the local table in (1). Compose it with the cumulative table once, across all original generators. If the current image of x_i is a^{k_i} and the local map sends a to z^f , its new image is $z^{k_i f}$. Mark the selected source slots deleted and remove the eliminated live labels. All donor checks precede any table mutation.

The reference producer scans source slots in fixed order, greedily choosing disjoint eligible pairs. An optional limit on pairs per batch supports controlled sequential schedules. On failure to find a pair it stops with a nondecisive endpoint; it does not call normalization or arbitrary search. At rank one it computes the signed exponent of every retained source root under the cumulative table. It reports the infinite-cyclic quotient endpoint only when all these exponents are zero.

Listing 1: Raw source-anchored producer; local operations are exact.

```
freeze source circuit, live labels, and original relator slots
```

```

images[i] := (i, 1); deleted := empty set
while number of live labels > 1:
    summaries := signed counts on source under current images
    batch := greedy coherent primitive-width donors in undeleted slots
    if batch is empty: return STALLED with the recorded prefix
    validate all batch witnesses against the pre-batch source images
    images := compose all original images with the disjoint local table
    delete precisely the chosen source slots; update live labels
check every remaining relator exponent at the one-generator endpoint
return recorded prefix and exact endpoint classification

```

Theorem 6.2 (Polynomial source-anchored epoch). *The reference producer completes one frozen-source epoch in polynomial bit time, including candidate discovery, witness construction, and the endpoint. Independent replay has a polynomial bound as well. The working word representation needs no new SLP nodes inside the epoch.*

For example, with $D = \Lambda + \lceil \log_2(M + m + \max X + 2) \rceil$, a deliberately loose schoolbook bound for production and replay is

$$O(r(m+1)(M+r+m)D^3). \quad (15)$$

Up to certificate storage, working data use $O((M+r+m)D)$ bits, with temporary candidate profiles discarded between scans.

Proof. There are at most $r - 1$ contractions and r candidate scans, including a possible final failed scan. A shared signed-count scan takes $O(M)$ arithmetic operations. Checking all m candidates separately by a full source profile scan takes $O(mM)$ operations; sharing these candidate-specific scans is not assumed. Updating the cumulative table takes $O(r)$ products per batch, and the terminal exponent pass takes $O(M + m)$ additions and tests. All integers involved have $O(D)$ bits by Theorem 4.1 and the source-length discussion. Quadratic multiplication and conservative cubic gcd accounting imply (15), with ample slack for loop and identifier costs. Replay checks at most $r - 1$ witnesses and has no more expensive task.

The count or profile assigned to each source node has constant arity, and only one candidate profile need be retained at a time. Images and slot metadata are linear in $r + m$. A certificate contains at most $r - 1$ witnesses, so its scalar and label payload is also polynomial. Structural dictionaries can be replaced by ordered maps if a deterministic comparison-model bound is preferred; the Python implementation uses ordinary dictionaries. $\square$

The bound is not advertised as an optimized exponent of input size. It establishes that raw phase count is no longer an exponential parameter. Practical dispatch, caching, and multi-candidate evaluation can be investigated without changing the proof obligation. External cancellation callbacks are assumed constant-time in these bounds; their actual costs must otherwise be added. The default standalone run has no such callback.

6.4 Materialize only at a real representation boundary

A fresh ordinary binary SLP for the current presentation can be built by constructing signed powers for the r cumulative images, then mapping each source concatenation once. This uses

$$O(M + r\Lambda)$$

gates and polynomial bit work. An export may be necessary before calling an existing normalizer or another backend. It is unnecessary for the source-based primitive checks, and performing it after every raw batch defeats the main representation saving. The benchmark includes this eager policy as an explicit negative control.

Appending exports into an old arena rather than creating a fresh one retains history and must be charged accordingly. The supplied adapter does not free or rewrite the caller’s arena, certify an arbitrary image table, or silently change the existing source-provenance contract.

7 Independent replay and the nonabelian safety boundary

The package contains a producer and a separate verifier. They share the immutable source grammar definition, but the verifier imports neither the producer’s planner, state class, width routine, nor update helper. For each witness it traverses only nodes reachable from that source donor and computes signed counts and prefix extrema together. The producer instead performs full source scans and uses a unit-occurrence shortcut. This separates important algorithmic failure modes; it is not a claim of formal verification in a proof assistant.

7.1 Certificate contract

The top-level certificate contains exactly a format identifier, a source SHA-256 identifier, a list of disjoint batches, and an endpoint classification. Each witness records its original slot, ordered pair of live labels, primitive signed vector, positive power exponent, and width. Scalar arithmetic fields use canonical hexadecimal strings, avoiding decimal conversion limits on very large exact integers. Identifiers must be strict integers, not Booleans.

Before converting a supplied hexadecimal scalar to an integer, the checker bounds its string length using a conservative version of Theorem 4.1. It then requires canonical spelling and checks its bit length. The checker rejects reused or invalid slots, nonlive labels, nonprimitive vectors, noncoherent words, foreign letters, incorrect widths, extra fields, and overlapping pairs. All witnesses in a batch are verified against the intact pre-batch state, before any update is applied.

The source hash is a bookkeeping binding, not a probabilistic word-equality oracle. The checker receives the caller’s actual source and recomputes every mathematical assertion from its content; no positive algebraic conclusion is obtained by comparing two hashes alone. Source serialization and incoming payload sizes still need an outer input-byte allowance in a production host. The kernel’s cooperative work allowance is not a hard process-memory limit.

At rank one, the verifier independently recomputes every retained source relator’s exponent. A deleted slot is exempt only because its own donor witness was verified. A nonempty raw circuit with exponent zero is accepted at this endpoint, since a word in one generator is determined by its exponent. No reduction or expansion is required. A `stalled` record is merely a verified prefix with no cyclic endpoint; replay does not certify that every possible donor has been searched.

7.2 Why the lattice audit cannot authorize a quotient

Example 7.1 (A false positive for abelianization-only reasoning). Consider

$$H = \langle a, b \mid a^2b^3 \rangle.$$

Its source row is $(2, 3)$, with primitive integer kernel vector $(3, -2)$. The row rank, annihilation, primitive gcd, and cofactor identities all hold. Nevertheless H is not infinite cyclic: sending a to a transposition and b to a three-cycle gives a surjection onto the nonabelian group S_3 . The word a^2b^3 is therefore not primitive. Its profile with increments $3, -2$ has width 6, not the required $2 + 3 - 1 = 4$, and the implemented primitive checker rejects it.

This example rules out a tempting but invalid shortcut: selecting rows of A with a convenient integer kernel and treating their quotient as a free group. The matrix argument controls the size of *already valid* nonabelian operations. It does not discover valid operations from homology alone.

7.3 Proper powers and topological provenance

For a proper-power donor w^e with $e > 1$, replay proves the local primitive structure and records a torsion-freeness obligation. Without that obligation, it certifies the quotient obtained by additionally killing w , not an isomorphism with the supplied arbitrary presentation. The returned `needs_torsion_freeness` field cannot be ignored.

A full recognizer must independently reconstruct the group presentation from the validated classical knot source, replay every preceding presentation move, authorize proper-power elimination from the resulting invariant, and retain its established terminal interpretation. The new research format has no planar-diagram field or complete presentation-prefix language. It is therefore *not* a standalone knot certificate. In particular, presenting an abstract algebraic fixture to this module is not a test of unknot recognition on a diagram.

8 Reproducible validation

All measurements below come from completed runs retained in the bundle. They were performed on CPython 3.13.5 under Linux x86-64. The package is standard-library Python and does not require Regina, a native compiler, a computer-algebra system, or a numerical tolerance. Exact run-length word comparisons, rational elimination, and integer determinants are used where specified.

8.1 Unit and adversarial tests

The final suite has **38 passing tests**; the retained final run took 0.651 seconds. The tests include exact small-word width checks, signed trees, nonunit primitive vectors, proper-power obligations, arbitrary label sets, all-relator endpoints, source rebinding, malformed scalar encodings, incorrect widths, invalid slots, duplicate or overlapping pairs, Booleans in integer fields, cancellation, work and expansion limits, and protocol-level export. A producer-monkeypatch test makes producer helpers raise exceptions while independent replay still succeeds. Additional tests check the reachable-node recurrence, the character bound, unit images on a meridional character, and unused long source gates.

The preceding upstream synthesis README reports its own maintained-suite checkpoints [2]. Those are not rerun results of this delivery. We did not obtain a complete native checkout in the execution environment, and the complete Python recognizer, its knot corpus, and the Rust port were not executed here.

8.2 Exact audit counts

The completed audit uses seed 2026100801 and took 13.353 seconds. It contains:

Checked object	Completed count
Random signed-tree presentations	600
Prefix checks against independent cofactor kernels	3,393
Prefix comparisons with source-derived updater	3,393
Exact raw-root run-length word comparisons	26,037
Positive binary words of lengths 2 through 12	8,166
Signed profile cases	32,664
Independent complete prefix replays	2,728
Additional nonunit primitive fixtures	324

Random trees vary signs, branching, redundant slots, and batch sizes. Their prefixes are checked using a separate linear audit: rational row rank, $A_J K = 0$, primitive block geds, and exact maximal minors calculated using fraction-free determinants. The word-comparison oracle compares the full raw words through exact run-length lists, rather than merely comparing their free reductions or hashes. The finite audit does not prove the general size theorem; it tests the implementation against its consequences.

The binary-word audit uses an independent expected set consisting of rotations of a mechanical Christoffel period, repeated according to the gcd of the letter counts. Four sign choices are tested for every positive word. The additional nonunit fixtures use coprime count pairs $(2, 3)$, $(3, 5)$, $(5, 8)$, $(8, 13)$, and $(13, 21)$, their rotations and signs, with further eliminations and retained raw zero-exponent words.

8.3 Large powers without literal expansion

The capacity cases instantiate (7) with $q = 2^b$.

r	b	Exponent bits	Source nodes	Chained nodes	Proof bytes
12	64	705	743	2110	3,307
20	64	1,217	1,279	3686	8,035
24	128	2,945	3,019	8826	19,999
64	1024	64,513	64,707	—	1,038,327

Table 1: Completed capacity checks. The final row uses the anchored producer and independent source replay only; the dash is not a measured chained result. Proof bytes are the retained JSON serialization counts.

The largest case verifies a cumulative exponent of 64,513 bits, corresponding to an expanded power far beyond literal storage. Its 63 sequential contractions are permitted by the theorem; no shallow schedule is used. The certificate itself is about one megabyte, emphasizing that compressed computation is not synonymous with negligible proof size.

Retained unsuccessful experiment. An earlier audit attempted larger chained-update comparisons at $(r, b) = (16, 256)$, $(32, 512)$, $(64, 1024)$ and exceeded a 200-second process cap before writing

a completed report. That attempt supplies neither a pass nor a valid timing comparison. Its scope is recorded in `data/exploratory-timeout.txt`. The completed audit uses smaller paired capacity cases and runs only the anchored producer plus independent replay for the largest case. The table does not silently extrapolate the missing comparison.

9 Paired maintenance measurements and a negative result

The benchmark isolates a sharply defined workload: *maintenance of a supplied, preverified raw schedule*. Each timed call includes initialization, every scheduled update, and the exact one-generator endpoint. Schedule discovery and independent replay occur outside the timers. The reference updater is the inspected native `apply_projection` function copied verbatim into a module and executed on the supplied compatible research `Arena`; this is not the entire native word arena or recognizer.

The experiment uses seven algebraic families, five measured rounds, one warm-up per arm, and randomized arm order within a round. There are five arms: lazy source-anchored maintenance, an identical lazy A/A control, chained raw substitution, its identical A/A control, and source-anchored maintenance that exports a full circuit after every batch. Thus 175 measured calls and 35 warm-ups complete. The seed is 2026100802; total driver time is 7.405 seconds. The schedules use one pair per batch, with power parameter $b = 16$.

Family	r	Lazy (ms)	Chained (ms)	Eager (ms)	C/L	E/L
star	32	0.350	1.575	1.960	4.49	6.20
star	64	1.077	4.839	6.456	4.28	5.87
star	128	4.236	21.917	23.490	4.12	5.54
star	256	14.925	51.865	92.259	3.44	6.19
chain	16	0.200	4.114	9.666	22.41	48.42
chain	32	0.390	16.713	63.708	45.70	153.11
chain	64	1.443	79.373	497.437	53.23	348.82

Table 2: Supplied-schedule maintenance only. Times are marginal medians; C/L and E/L are medians of within-round chained/lazy and eager/lazy ratios. They need not equal ratios of marginal medians. No entry is an end-to-end recognition speedup.

On these fixtures, the paired chained/lazy ratios are 3.44–4.49 for stars and 22.41–53.23 for chains. Such ratios show that maintaining a monomial table can avoid real raw-circuit work. They do not establish the same improvement when donor discovery, independent proof replay, earlier filters, diagram simplification, or normalization dominate a complete call. The difference between the standalone arena and the full production arena is an additional limitation of the timing comparison.

The A/A ratios range approximately from 0.76 to 1.11. This is material noise, especially for small calls. Exact work counts and output equality support the mechanism, but the quoted wall-clock factors should not be treated as precise hardware-independent constants. The raw JSON retains the randomized order, all samples, controls, structural counters, and source hashes.

Eager export is consistently harmful here. The source-anchored arm that materializes a new circuit after every batch is slower than chained maintenance in every measured case. Its paired eager/lazy ratios range from 5.54 to 348.82. On the 64-generator chain, marginal median maintenance times are 1.443 ms for lazy source evaluation, 79.373 ms for chaining, and 497.437 ms

Family	r	Lazy A/A	Chained A/A	Chained nodes
star	32	0.772	0.763	99
star	64	1.013	0.851	163
star	128	0.820	1.107	291
star	256	0.978	0.981	547
chain	16	1.007	0.928	786
chain	32	0.934	1.007	1634
chain	64	1.041	0.949	3330

Table 3: A/A controls and allocated chained nodes. Submillisecond controls show appreciable timing noise; all retained samples are reported, not replaced by selected reruns.

for eager export. The optimization must therefore preserve the lazy representation across the epoch. Merely rebuilding from the source after every substitution is not the proposed improvement.

10 Integration plan for the maintained repository

The implementation is packaged for review without mutating the user’s repository. A suggested destination is a new research directory under `Topology/UnknotRecognition/fast/`, followed by a separately reviewed integration into the primitive-projection route. Existing defaults should not change on the basis of the algebraic timings above.

10.1 The minimal host interface

At a host-selected raw boundary, snapshot the reachable source circuit, its fixed relator slots, and live labels. Retain the source’s complete verified presentation prefix in the existing host certificate; the research snapshot alone does not replace it. Run source-based checks and table updates while all permitted moves remain monomial and source-slot preserving. Independently replay the arithmetic prefix before accepting any algebraic endpoint. Export once when a normalizer or another circuit consumer genuinely needs a standard SLP, then freeze a new source after the host’s verified transition.

The included `native_adapter.py` supplies snapshot and export functions using the inspected arena protocol. It does not expose a new native CLI flag, modify existing version-six or version-seven certificates, or prove a cross-version compatibility claim. An implementation can initially run it in shadow mode: compare current raw words and verdict-neutral metadata while leaving the old path authoritative.

10.2 Source pin and review checks

The copied updater is pinned to commit

7518823550fbc8c217bc8be0113fe52002e77465

with whole-file Git blob

0aff98067b485d285a9c8026fcac56ae9934edc0.

The helper `integration/check_native_source.py` compares that whole-file Git hash and the updater’s abstract syntax tree with a local checkout. It is supplied but was *not run* against a full checkout in this delivery. The full native word arena’s inspected blob is recorded separately in the provenance file. Evolving repository heads must not be assumed byte-identical to this pin.

Before production adoption, a native integration should pass the entire maintained suite, source-reconstructing certificate replay, adversarial resource tests, and paired whole-call measurements on actual knot diagrams. It should cover raw-to-normalized and normalized-to-raw transitions, unchanged legacy certificate formats, proper-power authorization, cancellation during export, and all-relator terminal checks. A memory audit must account for the host’s old arena, the frozen source, live tables, proof buffers, and temporary scans simultaneously.

10.3 Suggested changes to the maintained exposition

The earlier raw-phase bounds can be retained as elementary coarse bounds and followed by Theorems 4.1 and 5.2. The text should then replace “a short raw schedule is needed to control repeated exponent growth” by “raw epochs have polynomial encoded cost; discovery, representation resets, and decisive completion require separate control.” The forest discussion can invoke Lemma 2.3 rather than compounding an r factor per forest phase indefinitely.

This is a refinement to the analysis, not evidence that the old code was incorrect. Nor does a polynomial raw epoch justify enabling primitive projection or forests by default: the existing measurements report ordinary corpus overhead in some configurations [3, 4]. The new experiment does not supersede those whole-call results.

11 A more precise route to quasi-polynomial recognition

Write QP for time $\exp((\log(n+2))^{O(1)})$ on explicit input size n . The new local theorem supports a cleaner conditional composition statement.

Theorem 11.1 (Epoch-search composition). *Suppose a source-correct recognition procedure on an n -bit knot input explores at most $\exp(O(\log^\alpha(n+2)))$ states, for a fixed $\alpha \geq 1$. Assume each state has a frozen-source presentation, initial numeric metadata, and live-label encoding of size at most $\exp(O(\log^\beta(n+2)))$, for a fixed $\beta \geq 1$. Suppose every nonraw transition, including source reconstruction, exposure, normalization, and bookkeeping, costs at most $\exp(O(\log^\gamma(n+2)))$, for a fixed $\gamma \geq 1$.*

If the search is sound and complete with correctly verified terminal tests, then replacing each raw epoch by either the analyzed maintained mechanism or the source-anchored algorithm gives total time

$$\exp\left(O\left(\log^{\max\{\alpha,\beta,\gamma\}}(n+2)\right)\right).$$

No separate logarithmic raw-phase-depth hypothesis is required.

Proof. Each raw epoch costs a fixed polynomial in its initial encoded parameters by Theorem 5.2 or 6.2. A polynomial of $\exp(O(\log^\beta(n+2)))$ has the same asymptotic form. Multiply the maximum per-state processing and transition cost by the state bound. Adding the corresponding logarithmic exponents yields the claimed maximum-degree bound. Soundness and completeness are supplied by the hypothesized search, not by this running-time multiplication. $\square$

The theorem isolates, rather than proves, the global obligations. In particular, it does not bound the number of states at which no coherent donor is exposed, the size of a new source after a nonmonomial move, or the cost of choosing successful moves. A raw producer can stall on a complicated presentation of $\mathbb{Z}$ without contradicting any local result. An exact exponential fallback still preserves completeness, but invoking it does not establish a quasi-polynomial worst-case bound.

A useful conceptual shift is to distinguish three quantities: current algebraic rank, encoded presentation size at epoch boundaries, and the size of the explored transition graph. Rank reduction controls the first inside a raw epoch. The cofactor argument controls intermediate monomial coefficients. Neither alone controls the other two quantities across arbitrary search.

12 Further research questions and concrete targets

The following problems prioritize missing global mechanisms and testable native improvements. None is presented as already resolved by the experiments.

- Q1. Bounded-cost exposure from knot provenance.** For source presentations actually reached from validated knot diagrams, can one expose a useful coherent primitive donor with a bounded family of nonmonomial moves? A meaningful theorem must bound both the number of tried moves and the encoded size of every resulting source, rather than merely showing that some sequence exists.
- Q2. An amortized potential across source resets.** Find a potential combining source grammar size, character height, selected-row rank, and presentation complexity that cannot increase too often under the host's normalization/exposure policy. The target is a bound on total reset cost, not another bound on contractions within a fixed epoch.
- Q3. Native source-anchored replay and complete-call measurement.** Implement a shadow backend preserving the existing source prefix and old certificate versions. Require exact equality of the full raw relator slots and paired timings including producer discovery, independent replay, export, and earlier pipeline stages on actual diagrams. Report regressions as well as speedups; do not use the supplied-schedule ratios as substitutes.
- Q4. Character-aware dispatch.** Use Theorem 4.4 to distinguish unit-image meridional epochs, nonzero-character blocks with small exponents, and zero-character blocks. Can a validated character predict whether source anchoring will save work before performing an expensive scan? The native cost model must include the cost of updating that character through the verified prefix.
- Q5. State-sensitive summaries with sublinear updates.** The reference implementation rescans the source when live labels merge. Can capped supports and occurrence counts be maintained incrementally with a provable amortized bound while allowing previously large supports to become small? Caching an old ineligibility marker is not a valid answer. Union-by-size or dependency indexing should be evaluated against full-scan costs and memory.
- Q6. Shared evaluation of many primitive widths.** Candidate profiles have different weights $(q, -p)$. Develop an exact multi-query representation for prefix extrema that avoids one full scan per candidate without producing an exponentially large parametric envelope. A negative envelope-size example would also clarify the right restricted source classes.

- Q7. Direct source-anchored forest certificates.** The theorem covers forests through linearization, but the research format contains only disjoint pair batches. Add direct forest replay with independently checked acyclicity, unit coordinates, and parent-first table composition. Compare certificate length and replay work against sequential linearization, including tall chains and shared-anchor stars.
- Q8. Sharper determinant bounds from knot-derived matrices.** The bound uses only entry size. Can sparsity, total unimodularity of particular prefix matrices, or controlled minors from the verified topological source replace $L^{r-1}r^{O(r)}$ by a smaller parameter? The meridional character already forces unit images before coordinate changes; the interesting question is what survives the actual exposure operations.
- Q9. Controlled extensions beyond monomial images.** Can a bounded-support generalization allow an original generator to map to a product of a few live powers while retaining a useful primitive lattice normalization and a polynomial representation invariant? A rational kernel bound alone will not control noncommutative word structure. Any extension needs new nonabelian arguments and explicit circuit-size bounds.
- Q10. A complete terminal class with a bounded transition graph.** Identify a diagram-derived class for which the exact native procedure reaches a verified terminal after quasi-polynomially many controlled exposure states. The class should be specified by checkable input properties, not by the existence of an unknown successful certificate of the desired size.
- Q11. Formalize the saturation and reachability lemmas.** The proof-assistant target can be small and precise: disjoint primitive columns, row-rank growth, the integer kernel identity, maximal-minor bounds, and the raw mapper’s reachable-node injection. Formalization should expose which claims need source provenance and which are pure integer linear algebra.
- Q12. Geometric interfaces to hierarchy compression.** Determine whether the polynomial raw-epoch component can serve as an exact local operation in a quantified handle- or hierarchy-based recognizer. A successful connection must construct the algebraic source from the geometric state and control the transition graph and attachment data; analogy alone is not an end-to-end complexity proof.

13 Conclusion

The source exponent-sum matrix constrains a whole sequence of primitive projections much more strongly than a recurrence on the latest word length. Selected source rows have exactly the rank lost by the quotient, and the cumulative monomial columns form their saturated integer kernel. This yields a phase-independent cofactor bound and, together with reachable-node accounting, a polynomial bound for complete raw epochs, even for the existing updater. The new source-anchored implementation further reduces unnecessary representation maintenance while preserving exact independent arithmetic replay.

The result changes the location of the remaining complexity problem. A shallow raw schedule is no longer needed to prevent exponential encoded growth inside a fixed epoch. The unresolved general work is to control exposure, new source sizes, reset costs, and the number of search states needed to reach a decisive terminal. The supplied proofs, counterexamples, tests, and retained negative measurements make that boundary explicit and provide a reviewable next step for the maintained recognizer.

A Proof of the coherent-width certificate

We prove Proposition 6.1 without invoking word expansion in the algorithm. Expansion is used only as a mathematical object in this proof. Invert the two generators independently so that the word is positive in letters a, b . Let its counts be ep, eq with $\gcd(p, q) = 1$, and set $N = p + q$. The height increments are q on a and $-p$ on b .

A.1 The minimum strip forces a repeated period

Both increments are congruent to q modulo N . Thus after j letters the height is congruent to the initial height plus jq modulo N . Since $\gcd(q, N) = 1$, the first N heights have distinct residues. A strip of width $N - 1$ contains exactly N integer heights, so it contains exactly one representative of each residue.

Translate the strip to $\{0, 1, \dots, N - 1\}$. From height h , an a step stays in the strip exactly when $h \leq p - 1$; a b step stays in the strip exactly when $h \geq p$. These alternatives partition the strip. Hence the next letter is uniquely determined by the current height. After N steps the residue returns to its start, so the actual height and then the letter sequence repeat. The word is a literal e th power of a period of length N . That period has zero net height, hence exactly p letters a and q letters b .

A.2 The forced period is primitive

Use induction on $p + q$. The base $p = q = 1$ gives a cyclic shift of ab , which is primitive. If $p > q$, every b is followed by an a : after a b the height lies in $\{0, \dots, q - 1\}$, below p . Cyclically, every b is therefore preceded by an a as well, because two b 's cannot be adjacent. Group each such preceding a with its b , replacing ab by one new symbol c .

The vertices removed by this replacement are precisely those immediately after these a steps, at heights $p, \dots, p + q - 1$. Every strip height is visited once in the period, so the remaining heights are exactly $0, \dots, p - 1$. An unpaired a step still has increment q , while a new c step has increment $q - p = -(p - q)$. The new cyclic word has counts $p - q, q$, coprime and positive, and width $p - 1 = (p - q) + q - 1$. It is primitive in the free group on a, c by induction. The assignment $a \mapsto a, c \mapsto ab$ sends a free basis to a free basis of $F(a, b)$, so its original period is primitive as well.

If $q > p$, interchange the letters and reverse the sign of height, reducing to the preceding case. Cyclic rotations and independent generator inversions preserve primitivity. This proves the certificate in its signed form.

A.3 Why circuit extrema suffice

A repeated homogeneous run has monotone height; only its endpoints can be extrema. For a concatenation, all prefixes lie either in the left part or in the translated right part, giving (14). Induction over the source DAG computes exactly the same extrema as the conceptual expanded word. No approximation, probabilistic fingerprint, or truncation enters the certificate.

B A worked three-generator epoch

Consider

$$G = \langle x_1, x_2, x_3 \mid x_1^2 x_2^{-1}, x_2^3 x_3^{-1}, x_3 x_3^{-1} x_1 x_1^{-1} \rangle.$$

The source matrix is

$$A = \begin{pmatrix} 2 & -1 & 0 \\ 0 & 3 & -1 \\ 0 & 0 & 0 \end{pmatrix}.$$

The first contraction sends x_2 to x_1^2 ; the current second source slot is literally $x_1^6 x_3^{-1}$. Its contraction sends x_3 to x_1^6 . The final cumulative column is $K = (1, 2, 6)^T$.

The two selected rows have alternating maximal minors $(1, 2, 6)$, so their primitive kernel vector is exactly the cumulative image. The third source slot is never deleted by a redundancy guess. Its final raw word is $x_1^6 x_1^{-6} x_1 x_1^{-1}$, whose exponent is checked to be zero at rank one. This small example displays the three separate mechanisms: nonabelian donor verification, global integer size control, and all-relator terminal checking.

C Package map and reproduction commands

The bundle contains the standalone article, all Python source, unit tests, exact audit and timing drivers, retained raw JSON and text logs, source provenance, an integration note, and a hash manifest. No external font files, third-party preprint PDFs, or complete copied repository are redistributed. The only reproduced native implementation fragment is the attributed MIT-0 raw updater in `reference_update.py`.

Listing 2: Commands from the unpacked bundle root.

```
# Exact unit and adversarial tests (standard library only).
PYTHONPATH=src python -B -m unittest discover -s tests -v

# Recompute exact audits or paired maintenance results to NEW files.
python -B experiments/audit.py --output data/audit-rerun.json
python -B experiments/benchmark.py --output data/benchmark-rerun.json --rounds 5

# Generate a small algebraic source and its independently replayed proof.
python -B experiments/demo.py

# Regenerate article tables from the retained completed measurements.
python -B experiments/make_tables.py

# Rebuild the article; no BibTeX database is required.
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex

# After obtaining the pinned ProveIt checkout, check the copied updater.
python -B integration/check_native_source.py --checkout /path/to/ProveIt
```

The last command is a future integration check, not a completed result reported here. The demo proves an algebraic endpoint for a supplied presentation; it does not construct or recognize a knot diagram. Reruns should write new result files so that measured historical samples are not silently replaced.

References

- [1] V. Reshetnikov and contributors, *ProveIt: Topology/UnknotRecognition*, repository README, inspected 8 October 2026. Git blob 7d80dc0740b232d01c113e26c156e6ac2370d92b. <https://github.com/VladimirReshetnikov/ProveIt/tree/main/Topology/UnknotRecognition>.
- [2] ProveIt contributors, *UnknotRecognition synthesis README*, inspected 8 October 2026, including the primitive-planner integration checkpoint. Git blob 86884529940b9962345028fbf00349be8890d23d. <https://github.com/VladimirReshetnikov/ProveIt/blob/main/Topology/UnknotRecognition/synthesis/README.md>.
- [3] ProveIt contributors, *Independently replayed disjoint primitive-pair projections*, maintained synthesis section, inspected 8 October 2026. Git blob 3e03140c627ccb12d9df2e7615a49df351eabc16. https://github.com/VladimirReshetnikov/ProveIt/blob/main/Topology/UnknotRecognition/synthesis/primitive_projection.tex.
- [4] ProveIt contributors, *Contracting unit-coordinate primitive forests in one raw batch*, maintained synthesis section, inspected 8 October 2026. Git blob 86518c69bf58c69e502e75075453fa1451f2716c. https://github.com/VladimirReshetnikov/ProveIt/blob/main/Topology/UnknotRecognition/synthesis/primitive_forest.tex.
- [5] ProveIt contributors, *fast/fastunknot/primitive_projection.py*, commit 7518823550fbc8c217bc8be0113fe52002e77465, whole-file Git blob 0aff98067b485d285a9c8026fcac56ae9934edc0, MIT-0. The complete module was inspected; its raw updater is reproduced in the bundle.
- [6] ProveIt contributors, *fast/fastunknot/compressed_words.py*, same pinned commit as [5], Git blob bd68418e226813a7fa50dc87405c214970ddf50e. The initial 200 source lines, including raw concatenation, binary powering, and reachable-node traversal, were inspected.
- [7] ProveIt contributors, *fast/fastunknot/primitive_power.py*, same pinned commit as [5], Git blob dcad51e89a2bdd01c1d87a40ae66bf016d2df7ea. Exact coherent primitive-power width queries on maintained word circuits.
- [8] Oxford Mathematical Institute, *Marc Lackenby announces a new unknot recognition algorithm that runs in quasi-polynomial time*, 2 February 2021. Announcement following the UC Davis seminar: <https://www.maths.ox.ac.uk/node/38304>.
- [9] M. Lackenby, *Incompressible surfaces, hierarchies and unknot recognition*, arXiv:2607.23350v1, July 2026, especially Sections 9–10. <https://arxiv.org/abs/2607.23350>. Section 9 was also inspected in the PDF on printed page 34.
- [10] R. P. Osborne and H. Zieschang, *Primitives in the free group on two generators*, *Inventiones Mathematicae* **63** (1981), 17–24. <https://doi.org/10.1007/BF01389191>.
- [11] M. Lohrey and A. Weiß, *The Power Word Problem*, in MFCS 2019, Leibniz International Proceedings in Informatics **138** (2019), 43:1–43:15. <https://doi.org/10.4230/LIPIcs.MFCS.2019.43>.